

Here

$$i \equiv n - m = n - 1, n - 2, \dots, 2 \quad (11.3.15)$$

and the quantity σ ($|x|^2$ in our earlier notation) is

$$\sigma = (a_{i0})^2 + \dots + (a_{i,i-1})^2 \quad (11.3.16)$$

Choose the sign of $\sqrt{\sigma}$ in (11.3.14) to be the same as the sign of $a_{i,i-1}$ to lessen roundoff error.

Variables are thus computed in the following order: $\sigma, \mathbf{u}, H, \mathbf{p}, K, \mathbf{q}, \mathbf{A}'$. At any stage m , $\mathbf{A}$ is tridiagonal in its last $m - 1$ rows and columns.

No extra storage arrays are needed for the intermediate results. At stage m , the vectors $\mathbf{p}$ and $\mathbf{q}$ are nonzero only in elements $0, \dots, i$ (recall that $i = n - m$), while $\mathbf{u}$ is nonzero only in elements $0, \dots, i - 1$. The elements of the vector $\mathbf{e}$ are being determined in the order $n - 1, n - 2, \dots$, so we can store $\mathbf{p}$ in the elements of $\mathbf{e}$ not already determined. The vector $\mathbf{q}$ can overwrite $\mathbf{p}$ once $\mathbf{p}$ is no longer needed. We store $\mathbf{u}$ in row i of $\mathbf{a}$ and $\mathbf{u}/H$ in column i of $\mathbf{a}$. Once the reduction is complete, we compute the matrices $\mathbf{Q}_j$ using the quantities $\mathbf{u}$ and $\mathbf{u}/H$ that have been stored in $\mathbf{a}$. Since $\mathbf{Q}_j$ is an identity matrix from row and column $n - j$ on, we only need compute its elements up to row and column $n - j - 1$. These can overwrite the $\mathbf{u}$'s and $\mathbf{u}/H$'s in the corresponding rows and columns of $\mathbf{a}$, which are no longer required for subsequent $\mathbf{Q}$'s.

The routine `tred2`, given below, includes one further refinement. If the quantity σ is zero or "small" at any stage, one can skip the corresponding transformation. A simple criterion, such as

$$\sigma < \frac{\text{smallest positive number representable on machine}}{\text{machine precision}}$$

would be fine most of the time. A more careful criterion is actually used. At stage i , define the quantity

$$\epsilon = \sum_{k=0}^{i-1} |a_{ik}| \quad (11.3.17)$$

If $\epsilon = 0$ to machine precision, we skip the transformation. Otherwise we redefine

$$a_{ik} \text{ becomes } a_{ik}/\epsilon \quad (11.3.18)$$

and use the scaled variables for the transformation. (A Householder transformation depends only on the ratios of the elements.)

If the eigenvectors of the final tridiagonal matrix are found (for example, by the routine in the next section), then the eigenvectors of $\mathbf{A}$ can be obtained by applying the accumulated transformation

$$\mathbf{Q} = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdots \mathbf{P}_{n-2} \quad (11.3.19)$$

to those eigenvectors. We therefore form $\mathbf{Q}$ by recursion after all the $\mathbf{P}$'s have been determined:

$$\begin{aligned} \mathbf{Q}_{n-2} &= \mathbf{P}_{n-2} \\ \mathbf{Q}_j &= \mathbf{P}_j \cdot \mathbf{Q}_{j+1}, \quad j = n - 3, \dots, 1 \\ \mathbf{Q} &= \mathbf{Q}_1 \end{aligned} \quad (11.3.20)$$

Input for the routine below is the real symmetric matrix $\mathbf{A}$ stored in the matrix `z[0..n-1][0..n-1]`. On output, `z` contains the elements of the orthogonal matrix $\mathbf{Q}$. The vector `d[0..n-1]` is set to the diagonal elements of the tridiagonal matrix $\mathbf{A}'$, while the vector `e[0..n-1]` is set to the off-diagonal elements in its components 1 through `n-1`, with `e[0]=0`.

Note that when dealing with a matrix whose elements vary over many orders of magnitude, it is desirable that the matrix be permuted, insofar as possible, so that the smaller elements are in the top left-hand corner. This is because the reduction is performed starting from the bottom right-hand corner, and a mixture of small and large elements there can lead to considerable rounding errors.

In the limit of large n , the operation count of the Householder reduction is $4n^3/3$ for eigenvalues only, and $8n^3/3$ for both eigenvalues and eigenvectors. The routine `tred2` is designed for use with the routine `tqli` of the next section. `tqli` finds the eigenvalues and eigenvectors of a symmetric tridiagonal matrix. For many years, the combination of `tred2` and `tqli` was the most efficient known technique for finding all the eigenvalues and eigenvectors (or just all the eigenvalues) of a real symmetric matrix. For moderate-sized matrices, it is still competitive with newer, more complicated methods.

eigen_sym.h

```
void Symmeig::tred2()
```

Householder reduction of a real symmetric matrix `z[0..n-1][0..n-1]`. (The input matrix **A** to `Symmeig` is stored in `z`.) On output, `z` is replaced by the orthogonal matrix **Q** effecting the transformation. `d[0..n-1]` contains the diagonal elements of the tridiagonal matrix and `e[0..n-1]` the off-diagonal elements, with `e[0]=0`. If `yesvecs` is false, so that only eigenvalues will subsequently be determined, several statements are omitted, in which case `z` contains no useful information on output.

```
{
    Int l,k,j,i;
    Doub scale,hh,h,g,f;
    for (i=n-1;i>0;i--) {
        l=i-1;
        h=scale=0.0;
        if (l > 0) {
            for (k=0;k<i;k++)
                scale += abs(z[i][k]);
            if (scale == 0.0) {
                e[i]=z[i][l];
            }
            else {
                for (k=0;k<i;k++) {
                    z[i][k] /= scale;
                    h += z[i][k]*z[i][k];
                }
                f=z[i][l];
                g=(f >= 0.0 ? -sqrt(h) : sqrt(h));
                e[i]=scale*g;
                h -= f*g;
                z[i][l]=f-g;
                f=0.0;
                for (j=0;j<i;j++) {
                    if (yesvecs)
                        z[j][i]=z[i][j]/h;
                    g=0.0;
                    for (k=0;k<j+1;k++)
                        g += z[j][k]*z[i][k];
                    for (k=j+1;k<i;k++)
                        g += z[k][j]*z[i][k];
                    e[j]=g/h;
                    f += e[j]*z[i][j];
                }
                hh=f/(h+h);
                for (j=0;j<i;j++) {
                    f=z[i][j];
                    e[j]=g=e[j]-hh*f;
                    for (k=0;k<j+1;k++)
```

Skip transformation.

Use scaled a 's for transformation.
Form σ in h .

Now h is equation (11.3.4).
Store $\mathbf{u}$ in row i of $\mathbf{z}$.

Store $\mathbf{u}/H$ in column i of $\mathbf{z}$.

Form an element of $\mathbf{A} \cdot \mathbf{u}$ in g .

Form element of $\mathbf{p}$ in temporarily unused element of $\mathbf{e}$.

Form K , equation (11.3.11).
Form $\mathbf{q}$ and store in $\mathbf{e}$ overwriting $\mathbf{p}$.

Reduce $\mathbf{z}$, equation (11.3.13).

```

        z[j][k] -= (f*e[k]+g*z[i][k]);
    }
}
} else
    e[i]=z[i][1];
    d[i]=h;
}
if (yesvecs) d[0]=0.0;
e[0]=0.0;
for (i=0;i<n;i++) {
    if (yesvecs) {
        if (d[i] != 0.0) {
            for (j=0;j<i;j++) {
                g=0.0;
                for (k=0;k<i;k++)
                    g += z[i][k]*z[k][j];
                for (k=0;k<i;k++)
                    z[k][j] -= g*z[k][i];
            }
        }
        d[i]=z[i][i];
        z[i][i]=1.0;
        for (j=0;j<i;j++) z[j][i]=z[i][j]=0.0;
    } else {
        d[i]=z[i][i];
    }
}
}
}

```

Begin accumulation of transformation matrices.
This block skipped when $i=0$.

Use $\mathbf{u}$ and $\mathbf{u}/H$ stored in $\mathbf{z}$ to form $\mathbf{P}\cdot\mathbf{Q}$.

Reset row and column of $\mathbf{z}$ to identity matrix for next iteration.

Only this statement remains.

CITED REFERENCES AND FURTHER READING:

- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.1.[1]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.4 Eigenvalues and Eigenvectors of a Tridiagonal Matrix

We now turn to the second step in the grand strategy outlined in §11.2, namely computing the eigenvectors and eigenvalues of a tridiagonal matrix.

11.4.1 Evaluation of the Characteristic Polynomial

Once our original real symmetric matrix has been reduced to tridiagonal form, one possible way to determine its eigenvalues is to find the roots of the characteristic polynomial $p_n(\lambda)$ directly. The characteristic polynomial of a tridiagonal matrix can be evaluated for any trial value of λ by an efficient recursion relation (see [1], for example). The polynomials of lower degree produced during the recurrence form a Sturmian sequence that can be used to localize the eigenvalues to intervals on the